

RISC-V Mikrocontroller auf FPGA

Abschluss Forschungsmaster I

1 Einstieg

Thema

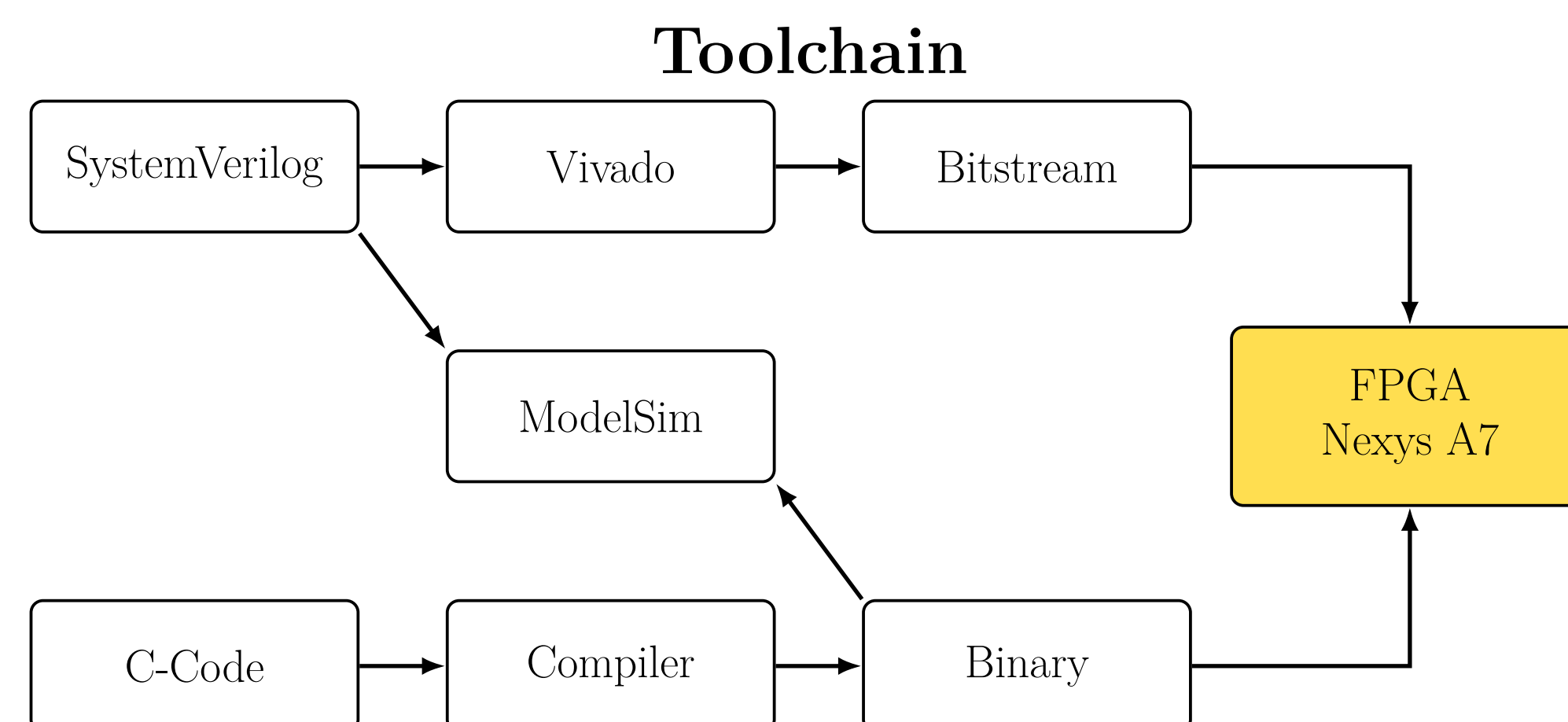
Ein Mikrocontroller ist der kleine Rechner in eingebetteten Geräten. **RISC-V** ist eine offene, lizenzfreie CPU-Architektur und gilt als Zukunftstechnologie. Das Projekt baut einen RISC-V-Mikrocontroller auf einem **FPGA** – einem frei programmierbaren Chip – der später für Lehrzwecke eingesetzt werden soll.

Voraussetzungen

- CPU-Kern: **LowRISC Ibex** (32-Bit RISC-V), unverändert übernommen.
- Drumherum (RAM, Register, Anzeige, Eingaben...) wird selbst in SystemVerilog entworfen.
- Plattform: **Digilent Nexys A7-100T** mit Xilinx Artix-7 FPGA.

Ziel

RISC-V und die zugehörige Toolchain als kompletten Lehrbaustein verfügbar machen – und darauf aufbauend ein eigenes, anspruchsvolles Peripheriegerät entwerfen.



2 Ergebnisse

Das erste Semester diente dem Ziel *Toolchain aufbauen und Grundlagen lernen* – SystemVerilog, FPGA-Workflow und das Ibex-Speicher-Interface. Alle dafür gesetzten Ziele wurden erreicht.

Umgesetzt:

- Eigene Bausteine: **RAM**, **memory-mapped Register**, **Tasten-Entpreller**, **Zähler**, **7-Segment-Anzeige**.
- FPGA-Demos auf der Nexys A7: Zähler → Anzeige, Zähler ↔ RAM → Anzeige, und Programme auf dem **Ibex-Core**.
- **C-Toolchain** (Linker-Skript, Startup-Code): der MCU lässt sich vollständig in C programmieren.
- **UART** als erste Peripherie – inkl. Demo.
- Jeder Baustein hat eine eigene Testbench mit **PASS/FAIL**-Ausgabe.

Offen verfügbar: Das gesamte Projekt (SystemVerilog-Bibliothek, Toolchain, Demos, Doku) ist öffentlich auf **GitHub**: github.com/tobiasbrandkulmbach-afk/smal_ibex_MCU_NexysA7100T

3 Zeitplan

Dieses Semester (Forschungsmaster I)

- Bibliotheksbausteine + Speicher-Demo **abgeschlossen**
- Ibex-Anbindung **abgeschlossen**
- **C-Toolchain** (Linker-Skript, Startup-Code), damit der MCU mit C programmiert werden kann **abgeschlossen**
- **UART** als wichtigste Peripherie **abgeschlossen**
- **Bootloader** für den eigenen Mikrocontroller **gestrichen**

Der **Bootloader** wurde **gestrichen**: Das Programm wird direkt über den FPGA-Bitstream geladen – für die Lehr-Toolchain kein Mehrwert.

Folgesemester (Forschungsmaster II)

- **Docker-Container** für die Toolchain, damit das Projekt auch an den FH-Rechnern ohne Einrichtungsaufwand läuft.
- Konzeption, Entwurf und Implementierung eines eigenen, anspruchsvollen Peripheriegeräts – voraussichtlich verschiedene Arten von **KI-Beschleunigern**, die miteinander verglichen werden.

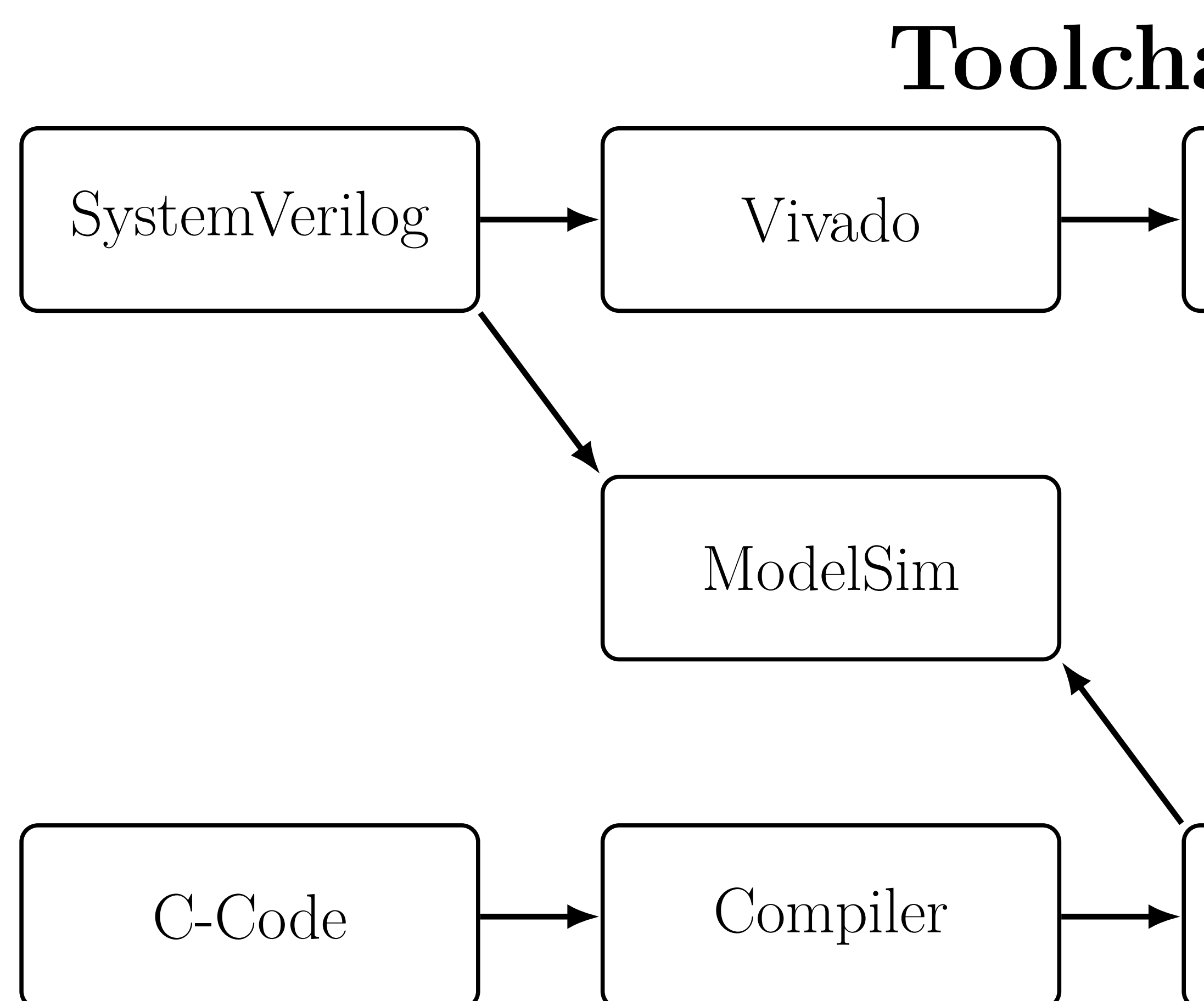
1 Einstieg

Thema

Ein Mikrocontroller ist der kleine Rechner in eingebetteten Geräten. als Zukunftstechnologie. Das Projekt baut einen RISC-V-Mikrocontroller der später für Lehrzwecke eingesetzt werden soll.

Voraussetzungen

- CPU-Kern: **LowRISC Ibex** (32-Bit RISC-V), unverändert übernommen.
- Drumherum (RAM, Register, Anzeige, Eingaben...) wird selbst in SystemVerilog entworfen.
- Plattform: **Digilent Nexys A7-100T** mit Xilinx Artix-7 FPGA.



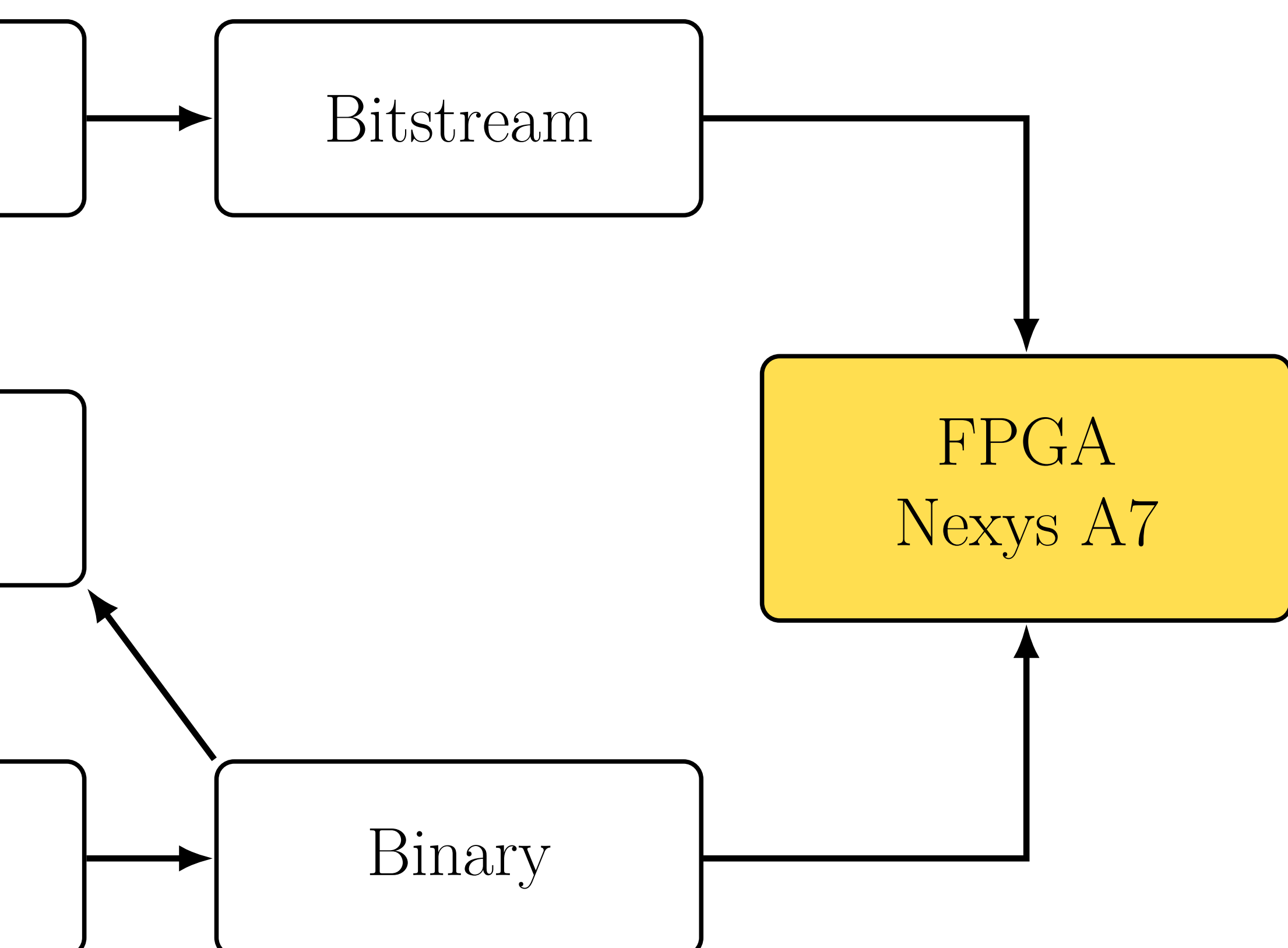
2 Ergebnisse

eräten. **RISC-V** ist eine offene, lizenzfreie CPU-Architektur und gilt
rocontroller auf einem **FPGA** – einem frei programmierbaren Chip –

Ziel

RISC-V und die zugehörige Toolchain als kompletten Lehrbau-
kasten verfügbar machen – und darauf aufbauend ein eigenes,
anspruchsvolles Peripheriegerät entwerfen.

Toolchain



3 Zeitplan

Projekt baut einen RISC-V-Mikrocontroller auf einem **FPGA** – einem m
gesetzt werden soll.

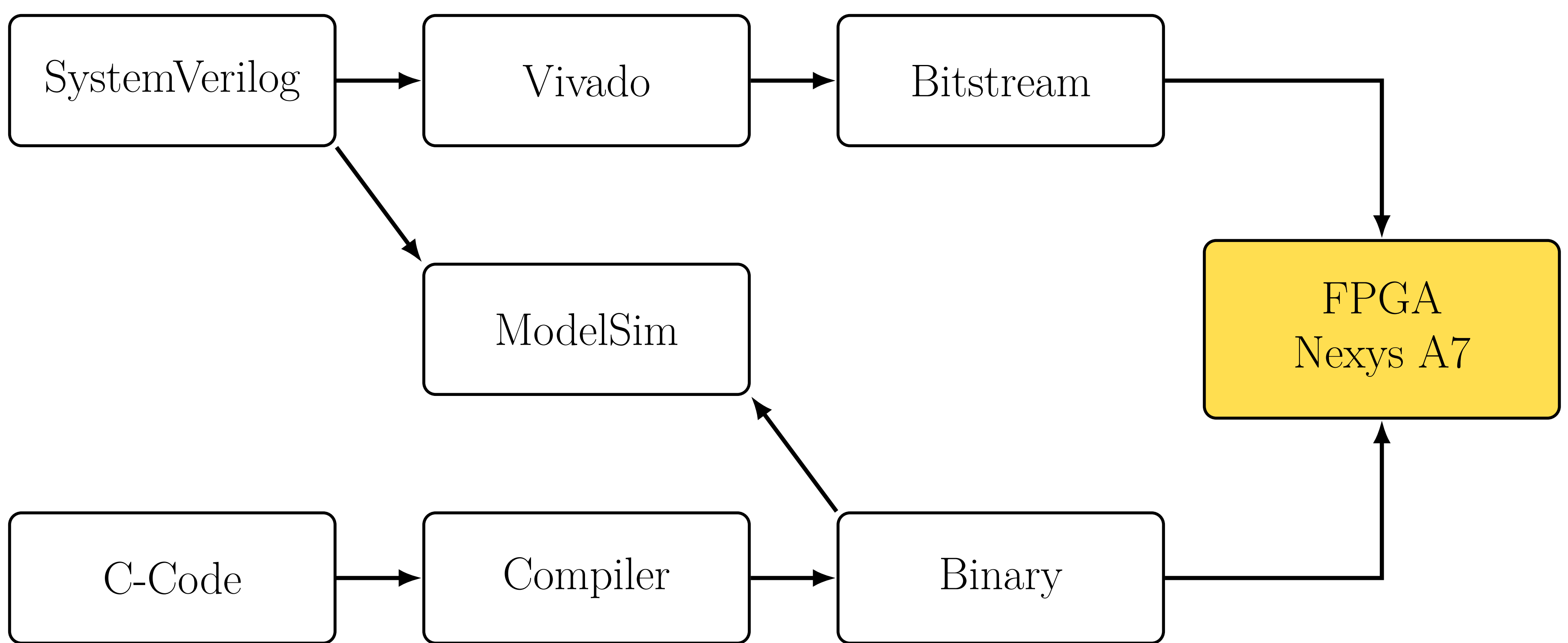
ibex (32-Bit RISC-V), unverändert
ster, Anzeige, Eingaben...) wird
worfen.

Nexys A7-100T mit Xilinx Artix-7

Ziel

RISC-V und die zugehörige Toolchain
kasten verfügbar machen – und da
anspruchsvolles Peripheriegerät ent

Toolchain



3 Zeitplan

dem Ziel *Toolchain aufbauen und*
mVerilog, FPGA-Workflow und das
dafür gesetzten Ziele wurden er-

, **memory-mapped Register**,
Zähler, **7-Segment-Anzeige**.

Nexys A7: Zähler → Anzeige, Zähler ↔

Dieses Semester (Forschung)

- Bibliotheksbausteine + Speicher-
- Ibex-Anbindung
- **C-Toolchain** (Linker-Skript, Sta
mit C programmiert werden kann
- **UART** als wichtigste Peripherie
- **Bootloader** für den eigenen Mi

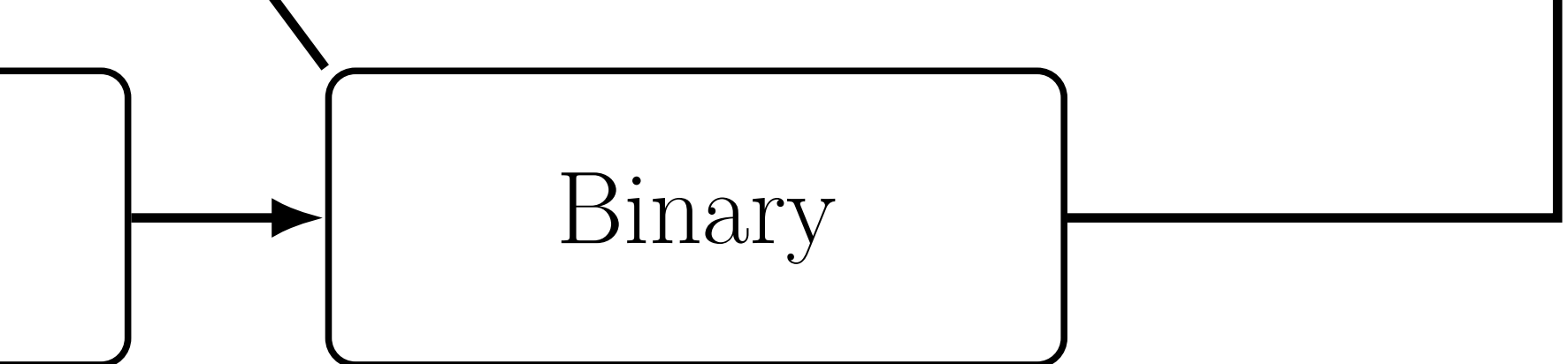
2 Ergebnisse

Das erste Semester diente dem Ziel *Toolchain aufbauen und Grundlagen lernen* – SystemVerilog, FPGA-Workflow und das Ibex-Speicher-Interface. Alle dafür gesetzten Ziele wurden erreicht.

Umgesetzt:

- Eigene Bausteine: **RAM**, **memory-mapped Register**, **Tasten-Entpreller**, **Zähler**, **7-Segment-Anzeige**.
- FPGA-Demos auf der Nexys A7: Zähler $\rightarrow$ Anzeige, Zähler $\leftrightarrow$ RAM $\rightarrow$ Anzeige, und Programme auf dem **Ibex-Core**.
- **C-Toolchain** (Linker-Skript, Startup-Code): der MCU lässt sich vollständig in C programmieren.
- **UART** als erste Peripherie – inkl. Demo.
- Jeder Baustein hat eine eigene Testbench mit **PASS/FAIL**-Ausgabe.

Offen verfügbar: Das gesamte Projekt (SystemVerilog-Bibliothek, Toolchain, Demos, Doku) ist öffentlich auf **GitHub**:
github.com/tobiasbrandkulmbach-afk/smal_ibex_MCU_NexysA7100T



Binary

3 Zeitplan

Dieses Semester (Forschungsmaster I)

- Bibliotheksbausteine + Speicher-Demo **abgeschlossen**
- Ibex-Anbindung **abgeschlossen**
- **C-Toolchain** (Linker-Skript, Startup-Code), damit der MCU mit C programmiert werden kann **abgeschlossen**
- **UART** als wichtigste Peripherie **abgeschlossen**
- **Bootloader** für den eigenen Mikrocontroller **gestrichen**

Der **Bootloader** wurde **gestrichen**: Das Programm wird direkt über den FPGA-Bitstream geladen – für die Lehr-Toolchain kein Mehrwert.

Folgesemester (Forschungsmaster II)

- **Docker-Container** für die Toolchain, damit das Projekt auch an den FH-Rechnern ohne Einrichtungsaufwand läuft.
- Konzeption, Entwurf und Implementierung eines eigenen, anspruchsvollen Peripheriegeräts – voraussichtlich verschiedene Arten von **KI-Beschleunigern**, die miteinander verglichen werden.

RISC-V Mikrocontroller auf FPGA

Abschluss Forschungsmaster I

1 Einstieg

Thema

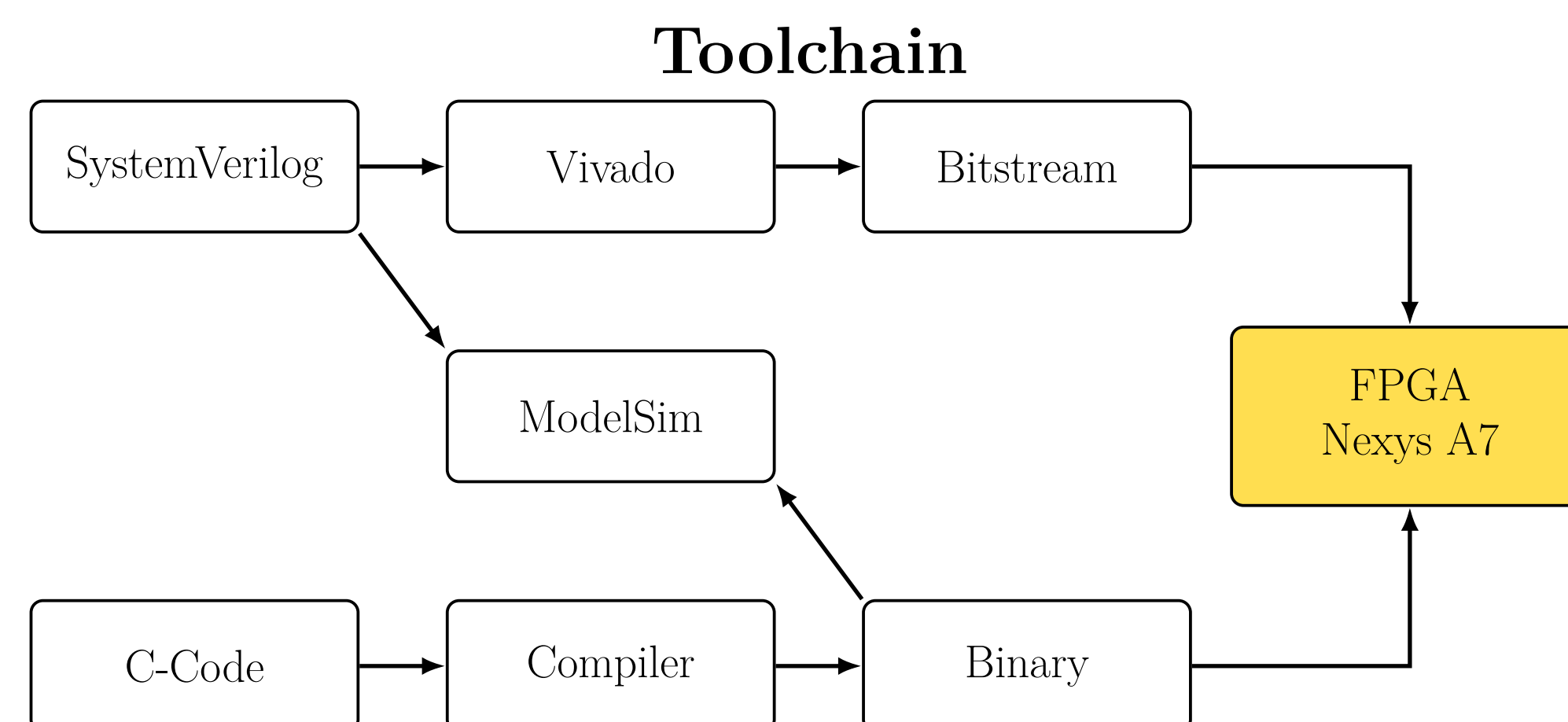
Ein Mikrocontroller ist der kleine Rechner in eingebetteten Geräten. **RISC-V** ist eine offene, lizenzfreie CPU-Architektur und gilt als Zukunftstechnologie. Das Projekt baut einen RISC-V-Mikrocontroller auf einem **FPGA** – einem frei programmierbaren Chip – der später für Lehrzwecke eingesetzt werden soll.

Voraussetzungen

- CPU-Kern: **LowRISC Ibex** (32-Bit RISC-V), unverändert übernommen.
- Drumherum (RAM, Register, Anzeige, Eingaben...) wird selbst in SystemVerilog entworfen.
- Plattform: **Digilent Nexys A7-100T** mit Xilinx Artix-7 FPGA.

Ziel

RISC-V und die zugehörige Toolchain als kompletten Lehrbaustein verfügbar machen – und darauf aufbauend ein eigenes, anspruchsvolles Peripheriegerät entwerfen.



2 Ergebnisse

Das erste Semester diente dem Ziel *Toolchain aufbauen und Grundlagen lernen* – SystemVerilog, FPGA-Workflow und das Ibex-Speicher-Interface. Alle dafür gesetzten Ziele wurden erreicht.

Umgesetzt:

- Eigene Bausteine: **RAM**, **memory-mapped Register**, **Tasten-Entpreller**, **Zähler**, **7-Segment-Anzeige**.
- FPGA-Demos auf der Nexys A7: Zähler → Anzeige, Zähler ↔ RAM → Anzeige, und Programme auf dem **Ibex-Core**.
- **C-Toolchain** (Linker-Skript, Startup-Code): der MCU lässt sich vollständig in C programmieren.
- **UART** als erste Peripherie – inkl. Demo.
- Jeder Baustein hat eine eigene Testbench mit **PASS/FAIL**-Ausgabe.

Offen verfügbar: Das gesamte Projekt (SystemVerilog-Bibliothek, Toolchain, Demos, Doku) ist öffentlich auf **GitHub**: github.com/tobiasbrandkulmbach-afk/smal_ibex_MCU_NexysA7100T

3 Zeitplan

Dieses Semester (Forschungsmaster I)

- Bibliotheksbausteine + Speicher-Demo **abgeschlossen**
- Ibex-Anbindung **abgeschlossen**
- **C-Toolchain** (Linker-Skript, Startup-Code), damit der MCU mit C programmiert werden kann **abgeschlossen**
- **UART** als wichtigste Peripherie **abgeschlossen**
- **Bootloader** für den eigenen Mikrocontroller **gestrichen**

Der **Bootloader** wurde **gestrichen**: Das Programm wird direkt über den FPGA-Bitstream geladen – für die Lehr-Toolchain kein Mehrwert.

Folgesemester (Forschungsmaster II)

- **Docker-Container** für die Toolchain, damit das Projekt auch an den FH-Rechnern ohne Einrichtungsaufwand läuft.
- Konzeption, Entwurf und Implementierung eines eigenen, anspruchsvollen Peripheriegeräts – voraussichtlich verschiedene Arten von **KI-Beschleunigern**, die miteinander verglichen werden.